

Java Operator Precedence

a guide to Java's operator types,
their precedence, and associativity



Java Operators

Arithmetic Operators
Assignment Operators
Relational Operators
Logical Operators
Unary Operators
Bitwise Operators



Arithmetic Operators

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo (remainder after division)

used to perform
mathematical tasks on
variables and data



Arithmetic Operators

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo (remainder after division)

```
public static void main(String[] args) {  
    int x = 20;  
    int y = 8;  
  
    // Addition  
    System.out.println("x + y = " + (x + y)); // 28  
  
    // Subtraction  
    System.out.println("x - y = " + (x - y)); // 12  
  
    // Multiplication  
    System.out.println("x * y = " + (x * y)); // 160  
  
    // Division  
    System.out.println("x / y = " + (x / y)); // 2  
  
    // Modulo  
    System.out.println("x % y = " + (x % y)); // 4  
}
```



Assignment Operators

used to assign
values to variables



Operator	Example
=	<code>x = y;</code>
+=	<code>x += y;</code>
-=	<code>x -= y;</code>
*=	<code>x *= y;</code>
/=	<code>x /= y;</code>
%=	<code>x %= y;</code>



Assignment Operators

Operator	Example
=	x = y;
+=	x += y;
-=	x -= y;
*=	x *= y;
/=	x /= y;
%=	x %= y;

```
public static void main(String[] args) {  
  
    // First Example  
    int integer = 10;  
  
    // Second Example  
    integer += 20;  
    System.out.println(integer); // 30  
  
    // Third Example  
    integer *= 2;  
    System.out.println(integer); // 60  
  
    // Fourth Example  
    integer /= 4;  
    System.out.println(integer); // 15  
  
}
```



Relational Operators

check the relationship
between two operands



Operator	Meaning
==	Is Equal To
!=	Not Equal To
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To



Relational Operators

Operator	Meaning
<code>==</code>	Is Equal To
<code>!=</code>	Not Equal To
<code>></code>	Greater Than
<code><</code>	Less Than
<code>>=</code>	Greater Than or Equal To
<code><=</code>	Less Than or Equal To

```
public static void main(String[] args) {  
  
    int a = 15;  
    int b = 10;  
  
    System.out.println(a == b); // false  
    System.out.println(a != b); // true  
    System.out.println(a > b);  // true  
    System.out.println(a < b);  // false  
    System.out.println(a >= b); // true  
    System.out.println(a <= b); // false  
}
```



Logical Operators



used in decision making
to check whether an
expression is true or false

Operator	Example	Meaning
&&	exp1 && exp2	true only if both <i>exp1</i> and <i>exp2</i> are true
 	exp1 exp2	true if either <i>exp1</i> or <i>exp2</i> is true
!	!expression	true if <i>expression</i> is false



Logical Operators

```
public static void main(String[] args) {  
    System.out.println(1 < 2 && 3 > 4); // false  
    System.out.println(1 < 2 || 3 > 4); // true  
    System.out.println(!(1 < 2)); // false  
}
```

Operator	Example	Meaning
&&	exp1 && exp2	true only if both exp1 and exp2 are true
 	exp1 exp2	true if either exp1 or exp2 is true
!	!expression	true if expression is false



Unary Operators

used with one
operand only



Operator	Meaning
-	Unary Minus: inverts the sign of an expression
++	Increment: increments value by 1
--	Decrement: decrements value by 1



Unary Operators

Operator	Meaning
-	Unary Minus: inverts the sign of an expression
++	Increment: increments value by 1
--	Decrement: decrements value by 1

```
public static void main(String[] args) {  
  
    int x = 5;                                Prefix(++x) – increment  
                                              first, then return the value  
  
    // x is increased to 6  
    // then x is displayed  
    System.out.println(++x); // 6  
    System.out.println(x); // 6  
  
}
```

```
public static void main(String[] args) {  
  
    int x = 5;                                Postfix(x++) – return the  
                                              value first, then increment  
  
    // 5 is displayed  
    // then x is increased to 6  
    System.out.println(x++); // 5  
    System.out.println(x); // 6  
  
}
```



Bitwise Operators



used to perform
operations on
individual bits

Operator	Meaning
<code>~</code>	Bitwise Complement
<code><<</code>	Left Shift
<code>>></code>	Right Shift
<code> </code>	Bitwise OR
<code>&</code>	Bitwise AND
<code>^</code> (XOR)	Bitwise exclusive OR



Bitwise Operators

Operator	Meaning
<code>~</code>	Bitwise Complement
<code><<</code>	Left Shift
<code>>></code>	Right Shift
<code> </code>	Bitwise OR
<code>&</code>	Bitwise AND
<code>^</code> (XOR)	Bitwise exclusive OR

```
public static void main(String[] args) {  
  
    System.out.println(~2); // 0..010 -> 1..101 -> -3  
    System.out.println(2 << 1); // 0010 -> 0100 -> 4  
    System.out.println(2 << 2); // 0010 -> 1000 -> 8  
    System.out.println(8 >> 1); // 1000 -> 0100 -> 4  
    System.out.println(8 >> 3); // 1000 -> 0001 -> 1  
    System.out.println(5 & 3); // 0101 & 0011 -> 0001 -> 1  
    System.out.println(5 | 3); // 0101 | 0011 -> 0111 -> 7  
    System.out.println(5 ^ 3); // 0101 ^ 0011 -> 0110 -> 6  
  
}
```



Precedence

determines the order in which the operators
of a given expression are evaluated

expressions inside **parentheses ()** are
always evaluated first

use them for clarity when dealing with multiple operators



Precedence

$$10 + 20 * 4$$

$*$ higher
priority
than $+$

$$10 + (20 * 4)$$



$$(10 + 20) * 4$$



Precedence

10 + 20 * 4

* higher
priority
than +

10 + (20 * 4) ✓



Level	Operator	Description
16	()	parentheses
15	++ --	post-increment post-decrement
14	++ -- - ! ~	pre-increment pre-decrement unary minus unary logical NOT unary bitwise NOT
13	() new	cast object creation
12	* / %	multiplicative
11	+ -	additive and concat
10	<< >>	shift
9	< <= > >=	relational
8	== !=	equality
7	&	bitwise AND

6	^	bitwise XOR
5		bitwise OR
4	&&	logical AND
3		logical OR
2	? :	ternary
1	= += -= *= /= %=	assignment

1 < 2 || 3 < 4 && 5 > 6

1 < 2 || (3 < 4 && 5 > 6)

(1 < 2 || 3 < 4) && 5 > 6



Level	Operator	Description
16	()	parentheses
15	++ --	post-increment post-decrement
14	++ -- - ! ~	pre-increment pre-decrement unary minus unary logical NOT unary bitwise NOT
13	() new	cast object creation
12	* / %	multiplicative
11	+ -	additive and concat
10	<< >>	shift
9	< <= > >=	relational
8	== !=	equality
7	&	bitwise AND

1 < 2 && 3 < 4 && 5 > 6

1 < 2 && (3 < 4 && 5 > 6)

(1 < 2 && 3 < 4) && 5 > 6

6	^	bitwise XOR
5		bitwise OR
4	&&	logical AND
3		logical OR
2	? :	ternary
1	= += -= *= /= %=	assignment



Level	Operator	Description
16	()	parentheses
15	++ --	post-increment post-decrement
14	++ -- - ! ~	pre-increment pre-decrement unary minus unary logical NOT unary bitwise NOT
13	() new	cast object creation
12	* / %	multiplicative
11	+ -	additive and concat
10	<< >>	shift
9	< <= > >=	relational
8	== !=	equality
7	&	bitwise AND

10 + 20 - 30

Associativity

if two or more operators share the **same precedence**, the expression is evaluated according to its associativity: **left to right**, or **right to left**

6	^	bitwise XOR
5		bitwise OR
4	&&	logical AND
3		logical OR
2	? :	ternary
1	= += -= *= /= %=	assignment



Level	Operator	Description	Associativity
16	()	parentheses	left to right
15	++ --	post-increment post-decrement	not associative
14	++ -- - ! ~	pre-increment pre-decrement unary minus unary logical NOT unary bitwise NOT	right to left
13	() new	cast object creation	right to left
12	* / %	multiplicative	left to right
11	+ -	additive and concat	left to right
10	<< >>	shift	left to right
9	< <= > >=	relational	not associative
8	== !=	equality	left to right
7	&	bitwise AND	left to right

1
<
2
&&
3
<
4
&&
5
>
6

1
<
2
&&
(
3
<
4
&&
5
>
6
)

(
1
<
2
&&
3
<
4
)
&&
5
>
6

6	^	bitwise XOR	left to right
5		bitwise OR	left to right
4	&&	logical AND	left to right
3		logical OR	left to right
2	? :	ternary	right to left
1	= += -= *= /= %=	assignment	right to left



Practice Exercises

```
boolean x = 1 < 10 || 4 < 2 + 4;  
x = true
```

```
"x = 2 + 2 = " + 2 + 2;  
x = 2 + 2 = 22
```

```
int i = 1, j = 2, x = ++j + i++;  
x = 4
```

```
"x = 2 + 2 = " + (2 + 2);  
x = 2 + 2 = 4
```

```
int x = 9 / 3 * 4 ^ 2;  
x = 14
```

```
int d = 10, e = 4, c = 2,  
x4 = d++ - ++c - ++e;  
x = 2
```

```
x = false || true && false || true;  
x = true
```

```
int x, x = y = 2; x *= ++y + 5;  
x = 16
```



Key Takeaways

Java provides **six categories** of operators: arithmetic, assignment, relational, logical, unary, and bitwise

Precedence determines the order in which operators are evaluated in an expression

When operators share the same precedence, **associativity** (left-to-right or right-to-left) determines the evaluation order

Use **parentheses** for clarity – they always take the highest priority and make your intent explicit

No need to memorize the full table – most precedence rules follow **mathematical intuition**; refer to the table when in doubt

